1) Why do we need mean normalization in Collaborative Filtering?

Mean normalization is an important step in collaborative filtering because it helps to adjust the ratings given by users, making them more comparable and reducing biases. When users rate items, their rating scales can vary significantly. For example, one user might rate everything higher than another user, leading to skewed results. By normalizing the ratings, we can ensure that the collaborative filtering algorithm focuses on the relative preferences of users rather than their absolute ratings.

Imagine you have two friends who rate movies. One friend tends to give high scores, while the other is more critical. If you simply average their ratings, the first friend's scores might dominate the results, making it seem like a movie is better than it actually is. Mean normalization adjusts each user's ratings by subtracting their average rating from each of their scores. This way, the algorithm can better identify patterns and similarities in preferences, leading to more accurate recommendations.

2) How does mean normalization improve recommendation accuracy?

Mean normalization improves recommendation accuracy in collaborative filtering by addressing the following key aspects:

1. **Reduces Bias**: By subtracting the average rating of each user from their individual ratings, mean normalization removes the influence of a user's rating scale. This helps to ensure that the recommendations are based on relative preferences rather than absolute scores, which can vary widely among users.

2. **Enhances Similarity Measurement**: When calculating similarities between users or items, mean normalization allows for a more accurate comparison. It focuses on how much a user likes or dislikes an item relative to their typical ratings, making it easier to identify users with similar tastes.

3. **Improves Convergence**: In optimization algorithms used for collaborative filtering, mean normalization can lead to faster convergence. By centering the data around zero, it can help the algorithm to find optimal parameters more efficiently.

4. **Better Handling of Sparse Data**: In many recommendation systems, user-item interactions are sparse (i.e., most users have not rated most items). Mean normalization helps to mitigate the impact of this sparsity by focusing on the differences in ratings rather than the absolute values.

5. Mean normalization **does impact new user rating prediction** in collaborative filtering.

   Here’s how:

   1. **Cold Start User (no ratings yet):**

      1. Mean normalization can’t be applied (since no average rating exists).

      2. System often falls back to global average or item-based predictions.

2. **New User with Few Ratings:**
    1. Mean normalization helps by removing bias in their limited ratings.
    2. Predictions become based on *relative preferences* (how much more/less they liked something compared to their own average), making recommendations fairer.
3. **Effect on Predictions:**
    1. Without normalization → A harsh rater may always get low predictions.
    2. With normalization → The system adjusts for rating style, so predictions better reflect *true taste*.

👉 So, normalization is **beneficial once a user has at least some ratings**, but doesn't solve the **cold-start problem**.

By implementing mean normalization, collaborative filtering systems can provide more relevant and personalized recommendations, ultimately enhancing user satisfaction

3) What would happen if mean normalization is not applied?

If mean normalization is not applied in collaborative filtering, several issues can arise that negatively impact the accuracy and effectiveness of the recommendations:

1. **Bias in Recommendations**: Users who consistently rate items higher or lower than others can skew the results. For example, if one user rates all movies between 4 and 5 stars, while another rates them between 1 and 2 stars, the system may favor the first user's preferences, leading to biased recommendations.
2. **Inaccurate Similarity Measures**: Without mean normalization, the similarity calculations between users or items may be distorted. The algorithm may incorrectly identify users as similar based on their absolute ratings rather than their relative preferences, resulting in poor recommendations.
3. **Overemphasis on Rating Scale**: The absolute values of ratings can dominate the recommendation process. This means that a user's tendency to rate items highly could overshadow their actual preferences, leading to recommendations that do not align with their true tastes.
4. **Poor Performance with Sparse Data**: In many recommendation systems, user-item interactions are sparse. Without mean normalization, the algorithm may struggle to find meaningful patterns in the data, resulting in less effective recommendations.

5. **Reduced User Satisfaction**: Ultimately, the lack of mean normalization can lead to recommendations that do not resonate with users, reducing their satisfaction and engagement with the system.

In summary, not applying mean normalization can lead to biased, inaccurate, and less relevant recommendations, which can diminish the overall effectiveness of a collaborative filtering system.

Example of how **mean normalization really helps in prediction**.

🎬 **Example: 3 Users × 4 Movies**

We have ratings (1–5 scale, "–" = not rated):

User	Movie A	Movie B	Movie C	Movie D	Average
A	5	4	2	–	3.67
B	3	–	4	2	3.0
C	1	2	–	3	2.0

🔹 **Step 1: Without Normalization**

- User A looks generous (avg ≈ 3.67).
- User C looks strict (avg = 2).
- If we compare A's rating 5 vs. C's rating 1, it *seems* A loves movies way more.
- But maybe they just use different **scales** of rating.

- **Step 2: Apply Mean Normalization**

Formula:

$$\text{Normalized Rating} = \text{Actual Rating} - \text{User's Average}$$

- **User A (avg = 3.67):**
  - A: 5 – 3.67 = +1.33
  - B: 4 – 3.67 = +0.33
  - C: 2 – 3.67 = –1.67
  - D: –
- **User B (avg = 3.0):**
  - A: 3 – 3 = 0
  - B: –
  - C: 4 – 3 = +1
  - D: 2 – 3 = –1
- **User C (avg = 2.0):**
  - A: 1 – 2 = –1
  - B: 2 – 2 = 0
  - C: –
  - D: 3 – 2 = +1

- **Step 3: After Normalization Table**

User	Movie A	Movie B	Movie C	Movie D
A	+1.33	+0.33	–1.67	–
B	0	–	+1	–1
C	–1	0	–	+1

🔹 **Step 4: Interpretation**

Now we can see **patterns**:

- User A and B both rate **Movie C above their average**.
- User B and C both rate **Movie D above their average**.
- Movie A: User A rates it **much higher than average (+1.33)**, while User C rates it **lower than average (–1)** → shows real taste difference.

🔹 **Step 5: Predict Missing Rating (Example: User A, Movie D)**

To predict, we look at similar users (B & C).

- **User B for Movie D:** –1 (below average).
- **User C for Movie D:** +1 (above average).

Suppose similarity weights favor User C more → prediction for A's Movie D will be **slightly above average**.

Predicted Normalized Rating≈+0.5\text{Predicted Normalized Rating} \approx +0.5Predicted Normalized Rating≈+0.5

Add back User A's average (3.67):

Final Predicted Rating≈3.67+0.5=4.17\text{Final Predicted Rating} \approx 3.67 + 0.5 = 4.17Final Predicted Rating≈3.67+0.5=4.17

👉 So we predict **User A will rate Movie D around 4.2**.

✅ **Key Takeaway:**

- Without normalization → Predictions would be biased by harsh/generous raters.
- With normalization → We uncover true **relative taste patterns**, making predictions more accurate.

Do you want me to also show **side-by-side prediction with and without normalization** so you can clearly see the difference?

👍 Let's extend the **example** with a **side-by-side prediction** for **User A → Movie D** (the missing rating).

🎬 **Example Recap**

Ratings before normalization:

User	Movie A	Movie B	Movie C	Movie D	Average
A	5	4	2	–	3.67
B	3	–	4	2	3.0
C	1	2	–	3	2.0

We want to **predict User A's rating for Movie D**.

🔹 **Step 1: Prediction Without Normalization**

- User B gave **2** to Movie D.
- User C gave **3** to Movie D.
- Average of their ratings = (2 + 3) / 2 = **2.5**.

👉 So predicted rating for A would be ≈ **2.5**.

But notice ❌ — User A's average is 3.67 (generous rater). Predicting 2.5 is **too low** for A.

🔹 **Step 2: Prediction With Normalization**

From normalized table:

User	Movie A	Movie B	Movie C	Movie D
A	+1.33	+0.33	–1.67	–
B	0	–	+1	–1
C	–1	0	–	+1

- User B → Movie D = –1 (below his avg).

- User C → Movie D = +1 (above his avg).
- Average normalized = (–1 + 1) / 2 = **0**.

👉 Predicted normalized rating = 0.

Now add back User A's average (3.67):

Predicted Rating=3.67+0=3.67\text{Predicted Rating} = 3.67 + 0 = 3.67Predicted Rating=3.67+0=3.67

✅ Final Prediction = ≈ **3.7** (much more realistic for a generous rater like A).

🔹 **Step 3: Side-by-Side Comparison**

Method	Predicted Rating for A on Movie D
Without Normalization	**2.5** (too low, ignores rating style)
With Normalization	**3.7** (fair, respects A's generosity)

👉 **Conclusion:**
Mean normalization corrects rating bias. Without it, strict vs. generous users distort predictions. With it, predictions reflect **true relative preferences**.